

HPC2526 - Consegna 3: Progettazione di programmi paralleli

Prendiamo in considerazione 3 programmi serali in C che verranno parallelizzati nei prossimi laboratori: Calcolo di Pigreco (cpi) , Propagazione del calore (heat) e fattorizzazione RSA (factorize).

In questo laboratorio vedremo l'algoritmo utilizzato e per ciascuno analizzeremo gli aspetti progettuali della parallelizzazione e il loro impatto sulle performance:

- Legge di Amdahl: impatto della parte di codice non parallelizzabile
- Partizionamento del carico di lavoro della parte parallelizzabile (decomposizione di dominio o decomposizione funzionale)
- In caso di decomposizione di dominio (data-parallel):
 - valutare il metodo di decomposizione (1D, 2D, ..,) e la granularità (grana fine, grana grossa)
 - impatto sulla comunicazione
 - impatto sul bilanciamento del carico
 - impatto sulla gestione di thread/processi paralleli

Il tar Progettazione.tgz si trova nella consegna e sul cluster in

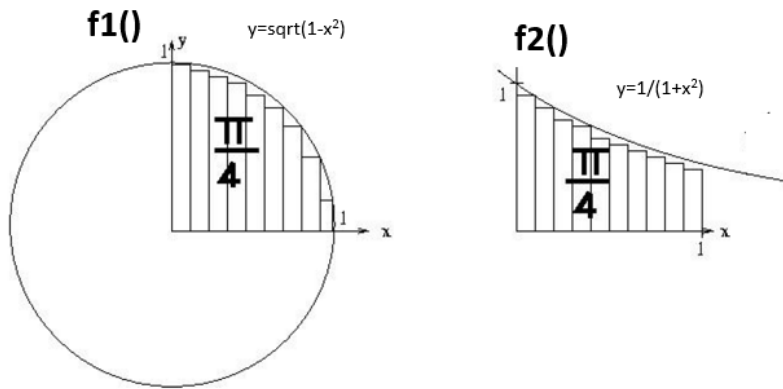
/hpc/home/roberto.alfieri/SHARE/HPC2526/

Per questo laboratorio gli script slurm possono essere eseguiti sia sul cluster (sbatch <script>.slurm) che sul PC (sh <script>.slurm).

Esercizio CPI

Il programma cpi calcola pi-greco utilizzando due diversi metodi di integrazione numerica.

Per il calcolo dell'integrale viene utilizzato il [metodo dei rettangoli](#) nell'intervallo tra 0 e 1.



Individuate nel programma `cpi` le funzioni **f1()** e **f2()** che implementano i due metodi, la funzione **getopt()** per la gestione dell'input dei parametri (selezione della funzione e del numero di intervalli) e **clock_gettime()** per determinare i tempi di calcolo.

L'opzione `-n` consente di selezionare il numero `n` di intervalli.

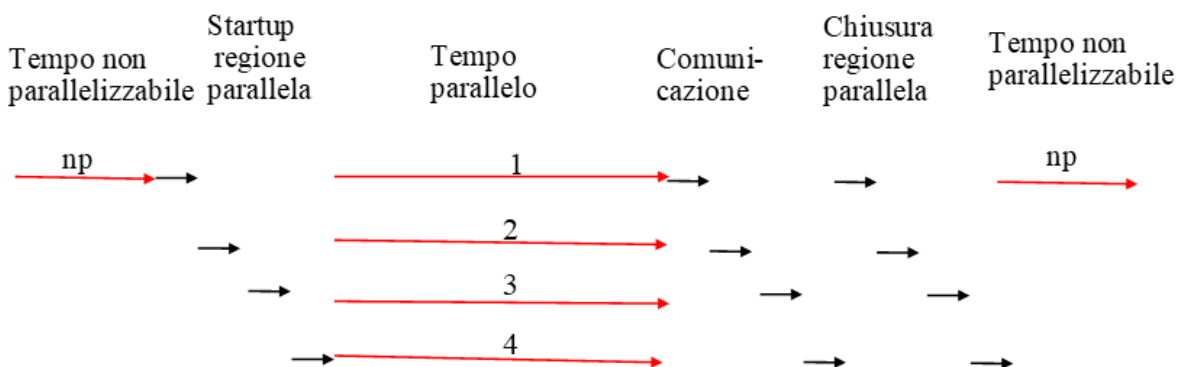
Il programma `cpi.c` utilizza per default il primo dei due metodi.

```
gcc cpi.c -o cpi -lm
./cpi -h
```

Parallelizzazione:

Un ciclo `for` ha il compito di calcolare tutte le aree degli intervalli tra 0 e 1. Le iterazioni sono indipendenti tra loro e possono essere distribuite tra i diversi thread o processi ed eseguite contemporaneamente. Un programma seriale può contenere parti di codice non parallelizzabili. Nel calcolo di Pigreco abbiamo una parte non parallelizzabile all'inizio per la gestione dell'input e alla fine per la gestione dell'output, ma entrambe le parti hanno un peso che diventa trascurabile al crescere del numero di iterazioni.

Programma parallelo



Opzionale: Il programma **cpi+amdahl.c** aggiunge la simulazione di una parte di codice non parallelizzabile tramite una chiamata `usleep` (parametrizzabile con `-s`). Editare `cpi_amdahl.py` inserire i valori di `tp` e `tnp` generati dell'esecuzione del programma, quindi generare il plot **cpi+amdahl.png**

Decomposizione del dominio. La parte parallelizzabile del programma (le iterazioni del ciclo `for`) può essere decomposta

- a grana grossa : dividiamo gli intervalli in P parti, dove P è il numero di processori
- a grana fine: dividiamo gli intervalli in piccoli blocchi (chunks) che vengono distribuiti alle unità di processamento a rotazione o su richiesta quando hanno completato il chunk precedente.

Comunicazione. Nel modello a memoria distribuita ogni processo accumula nella variabile locale `sum` i contributi dei cicli `for` che ha gestito. Al termine delle iterazioni tutti inviano la propria somma parziale al processo principale che somma tutti i contributi (operazione di **riduzione**).

Vengono inviati $p-1$ messaggi di 6 byte (un double) al termine del calcolo. Overhead trascurabile.

Sincronismo: Prima di comunicare i risultati e terminare i processi/thread occorre una barriera per attendere che tutti abbiamo completato il loro lavoro prima di fare la riduzione. Se usiamo decomposizione a grana fine non abbiamo tempi di idle. A grana grossa occorre fare attenzione che i sottodomini siano di pari dimensione o che non ci siano unità di processamento con diverse velocità.

Startup: prima di iniziare la regione parallela è necessario creare P processi/thread e eliminarli al termine. Operazioni svolte una sola volta, quindi trascurabili.

Tutti gli overhead sono trascurabili. Ci aspettiamo uno scaling ottimale dello speedup.

Esercizio heat

Il programma **heat.c** simula la propagazione del calore su una superficie rettangolare con alcune parti mantenute a temperatura costante (i bordi e il centro nella figura).

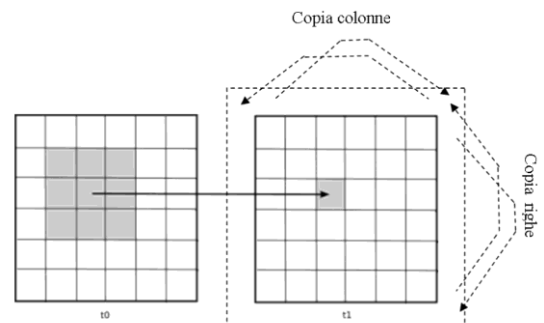


Individuate nel programma

- le funzioni `Init_center()` `Init_left()` e `Init_top()` che mantengono la temperatura nelle corrispondenti parti del rettangolo (parte centrale, bordo in alto e bordo a sinistra). Le funzioni `copy_rows()` e `copy_cols()` realizzano le **condizioni periodiche al contorno** nelle due direzioni, ovvero mettono a contatto termico i bordi alto/basso e destra/sinistra.
- la funzione `jacobi()` determina la nuova temperatura allo step `t1` come media della temperatura nei punti vicini allo step `t0` (in questo caso solo i 4 vicini, nord, sud, est e ovest). Il programma gestisce 2 matrici `t0` e `t1` che vengono scambiate (swap dei puntatori) al termine di ogni iterazione.

Fasi principali del programma:

```
for(liter=0; liter<MAX_ITER; liter=liter+1) {  
  // swap delle matrici;  
  // set delle temperature costanti;  
  // Jacobi iterator;  
}
```



Compilare ed eseguire il programma

```
gcc -O2 heat.c -o heat
```

Il programma scrive su stdout i valori della temperatura finale nei punti della superficie (heatmap) e su stderr i parametri e tempi di esecuzione.

L'opzione `-d` (debug) visualizza a schermo la dinamica dell'heatmap:

```
./heat -d -r 20 -c 20
```

Possiamo salvare la mappa del calore in un file `heatmap.txt`:

```
./heat 1>heatmap.txt 2>heat.csv
```

Opzionale: Lo script python **heatmap.py** utilizza la funzione `imsave()` per generare una color-map in formato png partendo dall'output del programma `heatmap.txt`. Modificare il programma in modo da ottenere qualche immagine della propagazione facendo variare le zone a temperatura costante con e

senza condizioni periodiche (commentare/decommentare il codice e ricompilare).

Opzionale: Lo script python **heatmap_animation.py** determina diverse heatmap al crescere del numero di iterazioni che poi utilizza per generare una animazione in **heatmap-animation.gif**. Le singole immagini png vengono incluse nel file gif in ordine alfabetico. Provare sul cluster a generare una animazione significativa. Il programma può funzionare anche sul PC ma occorre installare `imagemagick`.

Opzionale: Lo script **heat_scaling.slurm** sul PC o sul cluster genera un file csv in cui è rappresentato il tempo di esecuzione del programma (time) e del solo tempo nella funzione Jacobi (jtime) al crescere della dimensione della superficie. Eseguire lo script e generare il plot di scaling `heat_scaling.png`

Opzionale: Lo script `heat-pg.slurm`, che può essere eseguito sul PC e sul cluster, mostra come possiamo utilizzare il [profiler GNU](#) per determinare i tempi spesi nelle funzioni del programma. Scrivere i tempi di esecuzione delle funzioni coinvolte.

Considerazioni sulla parallelizzazione di `heat.c`

Decomposizione dominio: la funzione Jacoby, che ad ogni iterazione aggiorna la temperatura di tutti i punti della superficie, può essere parallelizzata distribuendo le righe della superficie tra i processi (memoria distribuita) o thread (memoria condivisa).

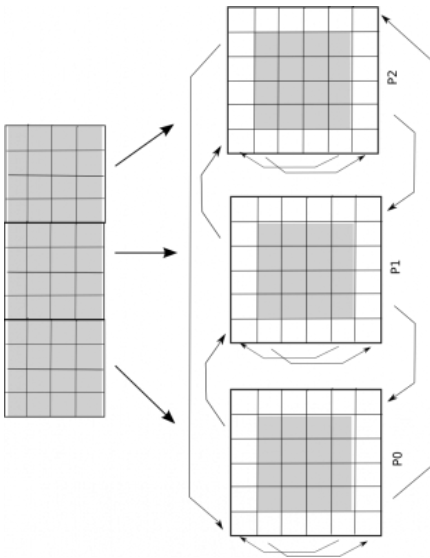
Ovviamente il ciclo for esterno `for(Iter=0; Iter<MAX_ITER; Iter=Iter+1)` non può essere parallelizzato poiché rappresentata la progressione temporale.

Per semplicità consideriamo tutto il resto non parallelizzabile (principalmente l'aggiornamento della temperatura nei punti caldi).

Overhead

Comunicazione. Quando si processa una riga occorre accedere anche ai dati della riga precedente e della riga successiva. Se la **memoria è distribuita** i processi che gestiscono blocchi di righe adiacenti devono scambiarsi le righe si bordo ad ogni iterazione; per questo motivo conviene in questo caso la decomposizione in sottodomini a grana grossa.

Se abbiamo `Iter` iterazioni e ogni riga contiene `N` punti ogni processo scambia 4 messaggi di `N float` ad ogni iterazione. $T_{comm} = 4 * Iter * T_{lat} * (N/Band)$

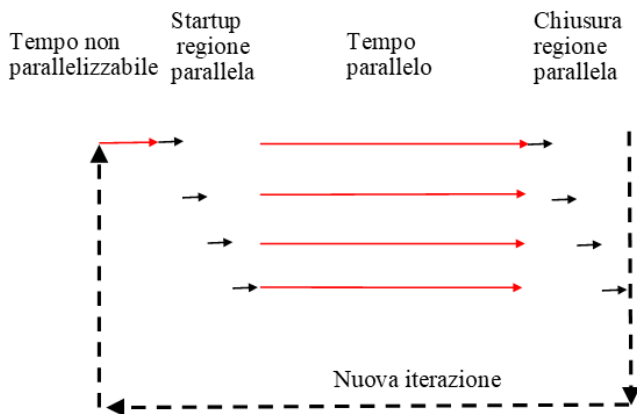


Se utilizziamo il modello a memoria condivisa i thread aggiornano parti diverse della stessa superficie. In questo caso possiamo operare anche a grana fine.

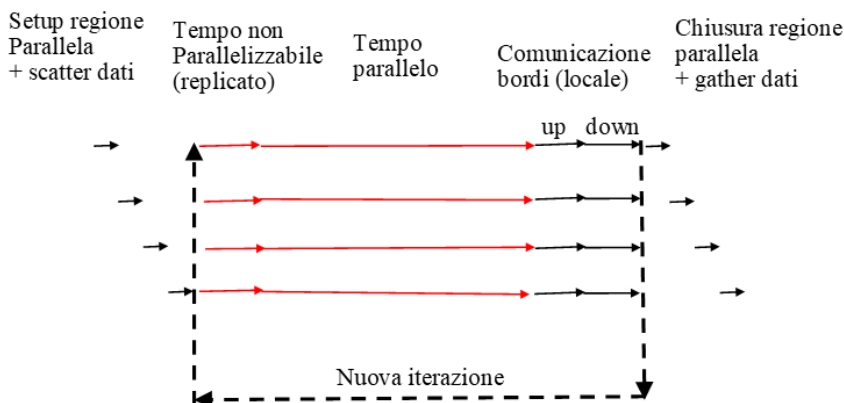
- **Sincronismo:** barriera (implicita o esplicita) al termine di ogni iterazione prima di inviare i messaggi. Se il dominio è decomposto in sottodomini della stessa dimensione non abbiamo tempi di Idle.

Il seguente diagramma riporta la profilazione temporale nell'esecuzione parallela.

MEMORIA CONDIVISA (multithread)



MEMORIA DISTRIBUITA (multiprocesso)



Esercizio Fattorizzazione

Il programma `factorize.c` utilizza la libreria BN (Big Numbers) di openssl per determinare due numeri primi grandi p e q , calcolarne il prodotto (Modulo). Il Modulo fa parte delle chiavi pubblica e privata dell'algoritmo crittografico RSA, che può essere violato se si riesce a scomporlo nei fattori primi p e q .

Il programma fattorizza a forza bruta, ovvero genera una sequenza di numeri primi p (dispari) e verifica il resto della divisione Modulo/p .

Per generare il Modulo si può utilizzare openssl:

```
openssl genrsa -out rsa_key.pem 512 # 512 è la lunghezza minima
openssl rsa -in rsa_key.pem -modulus -noout
```

Per generare un Modulo a scopo didattico con un numero limitato di bit usare il programma `factorize` con opzione `-g`. Esempio:

```
./factorize -g 68  
generating modulus of 68 bits  
P=03AAD0AEA7 Q=035BBF3587  
Modulus=0C50E77008F3CDAD11
```

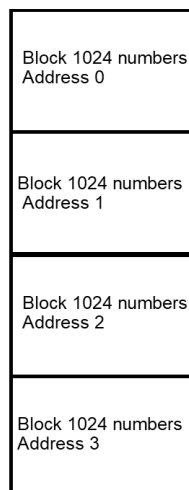
Lo spazio dei numeri primi da testare è suddiviso in blocchi di dimensione stabilita.

Ogni blocco ha un indirizzo (Address); il programma itera su gli indirizzi esaminando in sequenza tutti i blocchi:

```
for (i=0; i<maxAddress; i++) block_factorize(i);
```

Example:

```
Modulus      24 bit (2^24 numbers)  
Primes to test 12 bit (2^12 numbers)  
Block        10 bit size 1024  
Address      2 bit size 4
```



- Compilare ed eseguire il programma:
bash factorize.slurm # sul PC
sbatch factorize.slurm # sul cluster
- Studiare la tecnica ottimale di scomposizione del dominio e il relativo impatto della legge di Amdahl, e degli overhead introdotti dalla parallelizzazione (comunicazione, bilanciamento del carico, ecc) a memoria condivisa e distribuita.

Consegna: caricare i file di output (sul cluster `.<jobid>.out`), i csv e le immagini di scaling (.png) che avete creato e scrivere una breve relazione in `Relazione3.pdf` o `.txt`